

BOS: Dynamic Bottleneck-Oriented Scheduling for Parallel Machine Systems

Chen-Hong

Graduate Institute of Telecommunications and Communication
National Chung Cheng University, Taiwan
leon28473787@gmail.com

Abstract—Precedence-constrained scheduling on identical parallel machines becomes significantly more complex in multi-level Bill-of-Materials (BOM) systems due to asynchronous material arrivals, partial-kit execution, and dynamically shifting bottlenecks. These factors introduce time-dependent precedence relations that are not captured by classical models such as $P \parallel \text{prec} \parallel C_{\max}$.

This paper proposes BOS-Greedy, an event-driven list scheduling algorithm that prioritizes tasks using a dynamic bottleneck index, which captures the maximum overlap-adjusted downstream workload in the residual graph. Unlike classical list scheduling based on static critical paths, the proposed approach continuously tracks the evolving bottleneck structure under overlap-aware precedence activation.

Theoretical analysis establishes that BOS-Greedy achieves a $(2 - 1/m)$ -approximation ratio, extending Graham's classical bound to time-overlap precedence constraints via a generalized idle-time charging argument.

Experimental results on multi-level BOM instances demonstrate that BOS-Greedy matches HEFT under static conditions, achieves consistent makespan improvements as overlap allowances increase, and remains stable under dynamic bottleneck shifts.

These results indicate that BOS provides a deterministic, interpretable, and theoretically grounded scheduling framework for modern manufacturing systems.

Index Terms—Multi-level BOM scheduling, dynamic bottleneck, partial-kit activation, δ -overlap, event-driven scheduling.

I. INTRODUCTION

Scheduling in multi-level Bill-of-Materials (BOM) systems introduces challenges that are not captured by classical precedence-constrained scheduling models. In such environments, task dependencies are governed not only by logical precedence relations, but also by asynchronous material arrivals, partial-kit execution policies, and dynamically shifting bottlenecks.

Traditional models such as $P \parallel \text{prec} \parallel C_{\max}$ assume static precedence structures and synchronous task availability [1]. However, in practical manufacturing systems, operations may start before all predecessors are completed, provided that sufficient processing progress has been achieved. This behavior

leads to time-dependent precedence constraints that cannot be represented by classical AND-precedence models [2].

Advanced Planning and Scheduling (APS) systems play a central role in modern manufacturing by coordinating material availability, resource capacity, and precedence constraints. In practice, APS engines must repeatedly revise schedules in response to asynchronous material arrivals, bottleneck shifts, and partial-kit feasibility.

However, the commonly adopted practice of partial assembly introduces substantially greater scheduling complexity than assumed in traditional MRP frameworks [3]. When certain components arrive early, upstream operations may begin immediately, while critical components join only in later stages.

Thus, operation initiation is governed by a time-overlap activation condition: a task i may start only after each predecessor j has completed at least $p_j - \delta_{ji}$ units of processing, i.e.,

$$s_i \geq \max_{j \in \text{pred}(i)} (s_j + p_j - \delta_{ji}). \quad (1)$$

Actual start times are influenced by delay tolerance, discrepancies in material arrivals, and the depth of the BOM hierarchy. In this context, the effective critical path is often dictated by the slowest-arriving bottleneck component, and the bottleneck may shift over time due to supply delays, processing congestion, and capacity competition [2]. Consequently, the critical path is not a fixed longest-path structure [4] but instead evolves dynamically with the allowable overlap δ .

Scheduling with precedence constraints on parallel machines is NP-hard [5]. List Scheduling provides a fundamental approximation guarantee of $(2 - 1/m)$ for such problems [6], forming a cornerstone of classical scheduling theory. Yet prior work shows that local greedy decisions can lead to timing anomalies in multiprocessor environments, where adding resources paradoxically increases completion time [6]. Furthermore, the literature on Resource-Constrained Project Scheduling (RCPSP) indicates that, under resource contention, the effective completion time is governed by the resource-feasible critical chain, rather than the static longest path used by heuristics such as LPT [7]–[9]. Thus, the static longest-path perspective is insufficient in practical systems where bottlenecks are shaped by resource competition, material delays, and workload fluctuations [10], [11].

Approximation frameworks for shop scheduling—such as the ideal-schedule $\rightarrow$ random-delay $\rightarrow$ flattening

paradigm—offer polylogarithmic guarantees [12], [13], but they fail to capture cross-level dependencies in multi-level BOMs or asynchronous material arrivals [14]. Similarly, AND/OR precedence models allow partial predecessor completion but do not explicitly address lead times, hierarchical depth, or delay tolerance, making them unsuitable for multi-level manufacturing scenarios involving partial assembly [15]. Overall, existing literature lacks a unified scheduling framework that simultaneously captures: (i) deep BOM dependencies, (ii) dynamic bottleneck evolution, (iii) overlap-based initiation, and (iv) cross-operation interactions. While related works have studied precedence constraints and time-lag scheduling [16], [17], to the best of our knowledge, no existing approach simultaneously provides deterministic approximation guarantees under overlap-aware activation and dynamic bottleneck tracking.

In multi-level manufacturing processes, overall completion time is dictated by the dynamic behavior of the bottleneck chain rather than a static longest path. This behavior results from heterogeneous material arrivals, mid-process insertion of critical components, early operation initiation allowed by δ -tolerance, and shifts in bottlenecks caused by fluctuating machine loads [18]. Thus, future production scheduling and control must not rely solely on static rules; they must account for increasingly dynamic and uncertain manufacturing environments.

Although machine learning has been widely considered a promising enhancement to heuristic scheduling, recent surveys emphasize that such techniques cannot yet serve as full-scale replacements for complex decision-making algorithms [19]. Their performance depends heavily on specific data distributions or workload patterns and may degrade sharply under environmental shifts. Deep models also require extensive training data and costly offline exploration. Without approximation guarantees or feasibility assurances, learning-based scheduling can yield unpredictable or suboptimal outcomes in highly time-sensitive manufacturing settings—risking increased idle time, longer production cycles, or failure to meet delivery deadlines.

Furthermore, most learning-based scheduling or routing approaches lack interpretability, feasibility guarantees, and the theoretical performance bounds characteristic of classical algorithms. Therefore, learning-based approaches are better positioned as predictive auxiliary mechanisms rather than complete substitutes for structured scheduling theory.

Currently, no scheduling method simultaneously:

- captures dynamic bottleneck behavior,
- preserves the provable structure of list scheduling,
- supports δ -overlap initiation, and
- provides deterministic approximation guarantees.

Given these challenges, practical manufacturing systems require a deterministic approach that integrates: (1) structured decision logic, (2) verifiable approximation performance, and (3) explicit handling of overlap-based initiation and bottleneck-chain dynamics.

Our contributions are summarized as follows:

- We introduce an edge-wise time-overlap scheduling model that generalizes classical precedence constraints via explicit time-lag activation.
- We propose BOS-Greedy, an event-driven list scheduling algorithm that dynamically tracks bottleneck structures through an overlap-adjusted downstream metric.
- We establish a generalized $(2 - 1/m)$ -approximation bound by extending Graham’s classical analysis using a time-lag-aware idle-time charging argument.
- We demonstrate empirically that BOS-Greedy achieves robust performance under multi-level BOM settings with asynchronous material arrivals and dynamic bottleneck shifts.

II. RELATED WORK

Multi-level manufacturing scheduling involves hierarchical precedence constraints, multi-machine resource competition, cross-level material arrivals, and partial-kit activation. These characteristics cannot be fully captured by a single classical scheduling model. This section reviews four major bodies of literature to identify the theoretical limitations faced when modeling multi-level BOM structures and δ -overlap activation.

A. Problem Setting: Multi-Level BOM, Hierarchical Precedence, and Dynamic Bottlenecks

Modern manufacturing systems commonly employ multi-level Bills of Materials (BOMs) to represent product structures. The parent-child relationships in a BOM naturally form a directed acyclic graph (DAG) [20], [21]. A BOM distinguishes between independent demand for finished goods and dependent demand for subcomponents, and its deep hierarchical structure determines the permissible order of operations across production stages.

However, multi-level BOM scheduling differs fundamentally from classical precedence-constrained parallel-machine models such as $P \parallel \text{prec} \parallel C_{\max}$. Although List Scheduling provides a $(2 - 1/m)$ approximation bound [6], the model assumes that all tasks are known at time zero and depend solely on logical precedence relations. It does not incorporate material availability, supply lead times, or asynchronous cross-level component arrivals, and therefore represents a flat and static precedence graph.

In contrast, dependencies in multi-level BOMs arise from *material causality*. Higher-level assemblies cannot begin until all subcomponents have been procured, processed, and physically delivered. Thus, start times are shaped by real-world supply delays, queueing effects, processing load variations, and heterogeneous arrival patterns. This leads to a time-sensitive DAG whose behavior is driven by dynamic material availability.

Moreover, bottlenecks in multi-stage production shift over time. In classical scheduling, makespan is often determined by a fixed longest path [22]. However, in multi-level BOM systems, limiting factors may change due to material delays, machine congestion, or temporary queue buildups [23], [24].

Such *bottleneck shifting* implies that the critical chain is dynamic rather than static.

Overall, multi-level BOM scheduling is difficult because:

- operation start times depend on actual material arrivals rather than predetermined release constraints;
- delays in any component may propagate upward across layers;
- bottleneck locations evolve over time due to material flow variability and resource contention.

These characteristics do not exist in standard $P \parallel prec \parallel C_{max}$ models, indicating that multi-level BOM scheduling is not a direct subset of classical precedence-constrained scheduling.

B. Partial-Kit Activation, δ -Overlap, and Asynchronous Material Arrivals

A second key distinction from classical scheduling models is that many manufacturing systems allow operations to begin before all predecessors are completed. Industries such as semiconductor packaging, automotive electronics, and assembly manufacturing often adopt *partial-kit* policies [25]: once a subset of required components arrives, upstream operations start immediately to reduce wait time caused by late-arriving parts. This deviates from the traditional AND-precedence assumption that all predecessors must be completed.

This work formalizes such behavior using a tolerance parameter δ , representing the allowable degree of overlap between missing dependencies and operation initiation. Under δ -overlap activation, an operation may begin as long as the missing components do not block the executable portion. This is neither classical AND-precedence nor OR-precedence [15], but an intermediate dependency that allows limited overlap. The value of δ directly influences the feasible scheduling space.

Material arrivals in multi-level BOM systems are typically asynchronous. Heterogeneous arrival times cause some stages to begin early while others must wait, producing a hybrid execution pattern of “partially overlapping” and “partially blocked” operations.

Existing scheduling frameworks—including $P \parallel prec \parallel C_{max}$, RCPSP [7], and AND/OR models [7], [15]—assume strict activation rules that are not violated. None of them capture the behavior of “partial dependency satisfaction” allowed under δ -overlap. Therefore, current approximation algorithms cannot be directly applied to multi-level scheduling with partial-kit activation.

C. Dynamic Bottlenecks and Critical-Chain Deviation

In multi-stage manufacturing, makespan is not determined by a fixed longest path but by whichever production stage is currently constrained. Bottlenecks may shift due to temporary overloads, queue buildup, machine delays, or late materials [2]. These shifts occur rapidly and unpredictably, meaning the critical chain is not topologically predetermined but dynamically formed by system conditions.

As a result, any scheduling algorithm unable to detect and respond to bottleneck movement risks producing suboptimal or

incorrect decisions. Dynamic bottleneck shifting is a structural challenge that stems from multi-stage interactions and is independent of δ -overlap or arrival variability. Static precedence graphs cannot represent this behavior.

D. Classical Deterministic Scheduling Algorithms

1) *Method Categories*: Deterministic approximation algorithms for precedence scheduling can be grouped into five major categories:

a) 1) *Greedy / List Scheduling*: Simple, practical, and provides the $(2 - 1/m)$ bound [6]. However, it assumes static precedence and does not support asynchronous arrivals or changing ready sets.

b) 2) *HEFT (Heterogeneous Earliest Finish Time)*: A widely used DAG scheduler for heterogeneous processors [26]. Limitations include:

- all task information known at time zero,
- strict AND-precedence,
- static DAG structure,
- no support for δ -overlap or dynamic bottlenecks.

c) 3) *LP Rounding*: Offers strong worst-case guarantees such as 2-approx or PTAS [12], [27], but assumes all tasks can be modeled in a fixed precedence graph at time zero—invalid for BOM systems.

d) 4) *Primal–Dual / Dual-Fitting*: Effective for certain constrained settings but depends on static feasible domains [28].

e) 5) *Local Search*: Heuristically strong but lacks theoretical guarantees and assumes fixed topology [29].

2) *Representative Approximation Ratios*:

- Graham List Scheduling: $(2 - 1/m)$
- LP-based: 2-approx or PTAS
- Shmoys–Stein–Wein: polylogarithmic approximations
- Potts & Van Wassenhove: 2–3 approximation for batching and related settings

3) *Limitations of Classical Methods*: Classical approximation frameworks cannot handle:

- δ -overlap activation,
- asynchronous material arrival,
- dynamic bottleneck movement,
- cross-level delay propagation,
- multi-level BOM causal structures.

Thus, although theoretically mature, they are structurally incompatible with multi-level manufacturing requirements.

E. Machine-Learning-Based Scheduling

Recent ML-based scheduling approaches include reinforcement learning, supervised dispatching models, and learning-to-schedule frameworks [30]–[32]. They excel at pattern recognition and predictive tasks but suffer from:

- no performance guarantees (no approximation ratio),
- catastrophic degradation under workload shifts,
- lack of feasibility enforcement,
- inability to model material causality,
- inability to track bottleneck shifting.

Thus, ML is suitable as an auxiliary predictor, not a core scheduling mechanism for multi-stage BOM systems.

F. Positioning of BOS: Contributions Over Prior Work

The Bottleneck-Oriented Overlap Scheduling (BOS) method addresses the limitations identified above by providing:

- 1) a deterministic, interpretable scheduling rule built on analyzable list-scheduling structure;
- 2) a bottleneck-chain metric D_i extending longest-path reasoning to dynamic environments;
- 3) native support for multi-level BOM dependencies and material causal structure;
- 4) explicit modeling of δ -overlap activation;
- 5) event-driven recomputation capable of tracking dynamic bottleneck shifting;
- 6) a balance between theoretical analyzability and practical industrial feasibility.

BOS is therefore one of the few existing frameworks capable of jointly handling multi-level BOM structures, δ -overlap activation, and dynamic bottleneck behavior within a deterministic, approximation-aware scheduling paradigm.

G. Approximation Guarantee of BOS-Greedy

We establish a generalized performance bound extending Graham's classical list scheduling result to the time-overlap precedence model.

[Non-full interval path-cover] Let $I = [a, b)$ be any non-full time interval, i.e., an interval during which at least one machine is idle at all times. Then there exists a directed path P_I in the precedence graph such that

$$|I| \leq L_\delta(P_I) \leq L_\delta.$$

At any time $t \in I$, since the algorithm is work-conserving with respect to the executable set $R(t)$, the existence of an idle machine implies $R(t) = \emptyset$.

Thus, for every unfinished task i , there exists a predecessor $j \in \text{pred}(i)$ such that

$$t < s_j + p_j - \delta_{ji}.$$

Starting from any unfinished task, we can recursively trace blocking predecessors. Since the precedence graph is acyclic, this process yields a finite directed path.

By accumulating the time-lag constraints

$$s_i \geq s_j + p_j - \delta_{ji},$$

along this path and summing the inequalities, we obtain a telescoping bound showing that the total blocking duration is upper bounded by the overlap-adjusted path length $L_\delta(P_I)$.

Therefore, the duration of the non-full interval I is bounded by $L_\delta(P_I)$, and hence

$$|I| \leq L_\delta(P_I) \leq L_\delta.$$

[Active maximum-chain property] At any time t , if there exists an executable task belonging to a maximum overlap-adjusted critical chain, then BOS-Greedy schedules at least one such task.

By definition,

$$D_i(t) = \max_{P \in \mathcal{P}(i,t)} L_\delta(P),$$

which represents the maximum overlap-adjusted downstream workload starting from task i .

For any task i lying on a maximum L_δ -chain, the value $D_i(t)$ equals the maximum achievable downstream load among all tasks in the residual graph.

Since BOS-Greedy selects tasks according to

$$i^* = \arg \max_{i \in R(t)} D_i(t),$$

any executable task belonging to a maximum L_δ -chain will be selected whenever it is available.

Thus, at least one task on a maximum critical chain is always processed whenever such tasks are executable.

[Generalized Graham Bound for BOS-Greedy] For identical parallel machines, the makespan produced by BOS-Greedy satisfies

$$C_{\max} \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta,$$

where $W = \sum_{i \in V} p_i$.

Let $T = C_{\max}$ denote the makespan, and let I denote the total idle time across all machines.

We decompose total processing capacity as

$$mT = W + I.$$

We bound I using a charging argument.

Partition the time horizon into maximal non-full intervals. For each such interval I_k , by Lemma 1, there exists a directed path P_k such that

$$|I_k| \leq L_\delta(P_k) \leq L_\delta.$$

By Lemma 2, tasks on a maximum L_δ -chain are always processed whenever they are executable. Therefore, the maximum critical chain cannot remain entirely idle, and at most $(m-1)$ machines can be idle at any time.

Summing over all non-full intervals yields

$$I \leq (m-1)L_\delta.$$

Substituting into the capacity decomposition gives

$$mT \leq W + (m-1)L_\delta,$$

which implies

$$T \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta.$$

BOS-Greedy is a $(2 - 1/m)$ -approximation algorithm.

From the lower bound

$$C_{\max}^{\text{OPT}} \geq \max \left\{ \frac{W}{m}, L_\delta \right\},$$

and the upper bound

$$C_{\max}^{\text{BOS}} \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta,$$

TABLE I
TABLE I GAP ANALYSIS BETWEEN EXISTING SCHEDULING APPROACHES AND BOS-GREEDY.

Category	Existing methods	Limitations	BOS-Greedy
Scheduling paradigm	Static list-based scheduling	Delayed reaction to bottleneck shifts and asynchronous material arrivals	Event-driven scheduling with dynamic bottleneck tracking
Precedence handling	Strict AND-precedence constraints	No support for partial-kit execution	δ -overlap activation enabling controlled overlap
Theoretical grounding	Heuristic-based approaches	Lack of interpretable theoretical guarantees	Explicit alignment with the generalized Graham bound

we obtain

$$C_{\max}^{\text{BOS}} \leq \left(2 - \frac{1}{m}\right) C_{\max}^{\text{OPT}}.$$

From the lower bound:

$$C_{\max}^* \geq \max \left\{ \frac{W}{m}, L_{\delta} \right\},$$

and the upper bound:

$$C_{\max}^{\text{BOS}} \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_{\delta},$$

we obtain:

$$C_{\max}^{\text{BOS}} \leq \left(2 - \frac{1}{m}\right) C_{\max}^*.$$

III. PROBLEM FORMULATION AND THEORETICAL FOUNDATIONS

We consider a precedence-constrained scheduling problem on identical parallel machines under an edge-wise absolute time-overlap model.

A. Scheduling Model

Let $G = (V, E)$ be a directed acyclic graph (DAG), where each node $i \in V$ represents a task and each directed edge $(j, i) \in E$ represents a precedence constraint.

Each task i is associated with:

- processing time $p_i > 0$,
- release time $r_i \geq 0$.

A set of m identical machines is available. Each machine processes at most one task at a time, and each task requires exactly one machine. Tasks are processed non-preemptively.

For each edge $(j, i) \in E$, an overlap allowance $\delta_{ji} \in [0, p_j]$ is given. A feasible schedule must satisfy:

$$s_i \geq r_i, \quad (2)$$

$$s_i \geq \max_{j \in \text{pred}(i)} (s_j + p_j - \delta_{ji}). \quad (3)$$

The objective is to minimize the makespan:

$$C_{\max} = \max_{i \in V} (s_i + p_i). \quad (4)$$

a) *Remark.*: This is equivalent to a minimum start–start time-lag model with lag $d_{ji} = p_j - \delta_{ji} \geq 0$.

B. Overlap-Adjusted Critical Chain

For any directed path $P = (i_1, \dots, i_k)$, define:

$$L_{\delta}(P) = \sum_{t=1}^k p_{i_t} - \sum_{t=1}^{k-1} \delta_{i_t, i_{t+1}}. \quad (5)$$

Let:

$$L_{\delta} = \max_P L_{\delta}(P). \quad (6)$$

a) *Monotonicity.*: If $\delta'_{ji} \geq \delta_{ji}$ for all edges, then $L_{\delta'} \leq L_{\delta}$.

C. Basic Lower Bounds

Let $W = \sum_{i \in V} p_i$.

For any feasible schedule,

$$C_{\max} \geq \frac{W}{m}, \quad C_{\max} \geq L_{\delta}. \quad (7)$$

Thus,

$$C_{\max}^{\text{OPT}} \geq \max \left\{ \frac{W}{m}, L_{\delta} \right\}. \quad (8)$$

D. Residual Graph and Bottleneck Index

At time t , let $U(t)$ denote the set of unfinished tasks. Define the residual graph:

$$G(t) = (U(t), E(t)), \quad (9)$$

where $E(t) = \{(j, i) \in E \mid i \in U(t)\}$.

Define the dynamic bottleneck index:

$$D_i(t) = \max_{P \in \mathcal{P}(i, t)} L_{\delta}(P), \quad (10)$$

where $\mathcal{P}(i, t)$ denotes all directed paths starting from i in $G(t)$.

Equivalently,

$$D_i(t) = \begin{cases} p_i, & \text{if } \text{succ}(i, t) = \emptyset, \\ p_i + \max_{j \in \text{succ}(i, t)} (D_j(t) - \delta_{ij}), & \text{otherwise.} \end{cases} \quad (11)$$

E. Executable Task Set

A task i is executable at time t if:

$$t \geq r_i, \quad (12)$$

$$t \geq \max_{j \in \text{pred}(i)} (s_j + p_j - \delta_{ji}). \quad (13)$$

Let $R(t)$ denote the set of all executable tasks at time t .

F. Event-Driven BOS-Greedy Algorithm

We adopt an event-driven list scheduling framework.

Algorithm 1 BOS-Greedy

```

1:  $t \leftarrow 0, U \leftarrow V$ 
2: while  $U \neq \emptyset$  do
3:   Construct  $R(t)$ 
4:   Compute  $D_i(t)$  for all  $i \in R(t)$ 
5:   while there exists an idle machine and  $R(t) \neq \emptyset$  do
6:     Select  $i^* = \arg \max_{i \in R(t)} D_i(t)$ 
7:     Schedule  $i^*$  at time  $t$ 
8:     Remove  $i^*$  from  $R(t)$ 
9:   end while
10:  Let  $t$  be the earliest time of:
11:    (i) completion of a running task, or
12:    (ii) activation of a task satisfying precedence constraints
13:  Update  $U$ 
14: end while

```

a) *Work-Conserving Property.*: The algorithm is work-conserving with respect to the executable set $R(t)$, in the sense that whenever $R(t) \neq \emptyset$, no machine remains idle.

b) *Remark.*: This is a list scheduling variant in the sense of Graham [6], extended to time-overlap precedence constraints.

IV. EXPERIMENTAL DESIGN

A. Experimental Objectives

The goal of this study is not merely to demonstrate that BOS achieves lower makespan than existing heuristics, but to systematically evaluate its behavior from three complementary perspectives:

- **Theory alignment:** To verify whether the empirical performance of BOS is consistent with the theoretical lower bound derived from workload and overlap-adjusted critical chains.
- **Mechanism comparison:** To analyze how BOS differs from static-priority schedulers such as HEFT in terms of decision dynamics, particularly under overlap-aware precedence constraints.
- **Robustness across structures:** To evaluate the performance of BOS across diverse DAG structures, machine configurations, and overlap conditions, ensuring that results are not restricted to specific handcrafted scenarios.

B. Scheduling Models and Baselines

We evaluate BOS against two representative scheduling policies:

- **HEFT (Heterogeneous Earliest Finish Time):** A widely used static-priority list scheduling algorithm based on upward rank.
- **FIFO (First-In-First-Out):** A simple baseline that schedules tasks in the order of readiness.

All schedulers operate under the same execution model:

- Parallel identical machines
- DAG precedence constraints
- Edge-wise overlap allowance δ_{ij}
- Event-driven simulation (task completion, activation, machine release)

C. Instance Generation

To ensure comprehensive evaluation, we construct three types of instance families.

1) *Neutral Benchmark Families:* We generate synthetic DAGs with controlled structural properties, including:

- **Chain-heavy:** deep dependency chains
- **Fork-join:** parallel branches with synchronization points
- **Merge-heavy:** multiple predecessors per task
- **Overlap-rich:** high density of overlap allowances

Each instance is parameterized by:

- Number of tasks (small / medium / large)
- Machine count $m \in \{2, 4, 8\}$
- Overlap density and magnitude
- Processing time distributions

This design avoids bias toward any specific scheduling strategy.

2) *Mechanism-Oriented Cases:* In addition to neutral benchmarks, we construct controlled instances to highlight scheduling behavior under:

- Bottleneck shifts
- Overlap-induced precedence relaxation
- Late-stage critical task emergence

These cases are used for qualitative analysis rather than aggregate performance comparison.

3) *Small Exact Instances:* For small problem sizes, we compute optimal schedules using a branch-and-bound solver.

This enables direct evaluation of:

$$\text{gap}_{OPT} = \frac{C_{\max} - C^*}{C^*} \quad (14)$$

providing ground-truth validation of algorithm performance.

D. Performance Metrics

1) *Makespan:*

$$C_{\max} \quad (15)$$

The total completion time of all tasks.

2) *Lower Bound Gap:* We compute the theoretical lower bound:

$$LB = \max \left(\frac{W}{m}, L_\delta \right) \quad (16)$$

where W is the total workload and L_δ is the overlap-adjusted critical path length.

We report:

$$\text{gap}_{LB} = \frac{C_{\max} - LB}{LB} \quad (17)$$

3) *Optimality Gap*: For instances where the optimal solution is available:

$$\text{gap}_{OPT} = \frac{C_{\max} - C^*}{C^*} \quad (18)$$

4) *Decision Disagreement Rate*: To quantify behavioral differences between schedulers, we measure:

$$\text{disagreement}(A, B) = \frac{\# \text{ of differing decisions}}{\text{total decisions}} \quad (19)$$

This metric captures how frequently two algorithms select different tasks under identical system states.

5) *Idle Time Decomposition*: Idle periods are categorized into:

- Precedence-constrained blocking
- Release-time blocking
- Mixed blocking

This allows analysis of scheduling inefficiencies beyond makespan.

E. Experimental Protocol

For each configuration:

- Multiple random instances are generated
- Each scheduler is run independently
- Results are averaged over all instances

We report:

- Mean makespan
- Mean lower-bound gap
- Mean optimality gap (if available)
- Mean decision disagreement rate

All experiments are conducted using the same event-driven simulation framework to ensure consistency.

F. Implementation Details

The simulator is implemented in Python and follows an event-driven design:

- Task completion triggers updates to the residual DAG
- Bottleneck indices $D_i(t)$ are recomputed dynamically (for BOS)
- Scheduling decisions are made from the executable task set

For small instances, optimal schedules are computed using a branch-and-bound solver with pruning based on partial schedule bounds.

V. RESULTS AND ANALYSIS

A. Overall Performance Trends

This section analyzes the empirical behavior of BOS, HEFT, and FIFO under the benchmark settings described in Section IV. The results show three consistent patterns. First, BOS and HEFT achieve highly comparable makespan across most instance groups. Second, FIFO is consistently weaker than both BOS and HEFT, especially in medium and large structured instances. Third, the absolute performance gap between BOS and HEFT is generally small, indicating that

both methods operate in a regime close to the theoretical lower bound.

Figure 1 visualizes the relative makespan advantage of BOS over HEFT across all benchmark groups. The horizontal axis corresponds to different combinations of DAG family, instance size, and machine count, while the vertical axis reports the percentage improvement of BOS over HEFT. Most bars remain close to zero, showing that BOS does not consistently dominate HEFT in absolute makespan. However, this should not be interpreted as a weakness of BOS. Rather, it suggests that HEFT itself is already a strong baseline, and that the feasible improvement space is limited when the schedules produced by both methods are already near the theoretical lower bound.

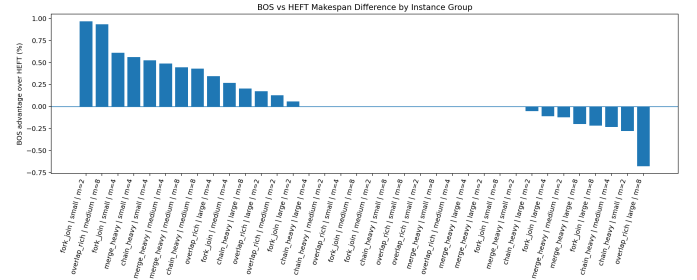


Fig. 1. Relative makespan advantage of BOS over HEFT across all benchmark groups. Positive values indicate that BOS achieves lower makespan, while negative values indicate that HEFT performs slightly better. Most differences remain close to zero, suggesting that both methods operate in a near-optimal regime.

To better understand whether the benchmark still distinguishes meaningful scheduling behavior, Figure 2 compares FIFO against BOS. Unlike the BOS–HEFT comparison, FIFO exhibits a clear penalty in many instance groups. This indicates that simple ready-order scheduling is insufficient under overlap-aware precedence constraints. In other words, the benchmark is not too easy; rather, it is sufficiently discriminative to separate structure-aware schedulers from naive dispatching rules.

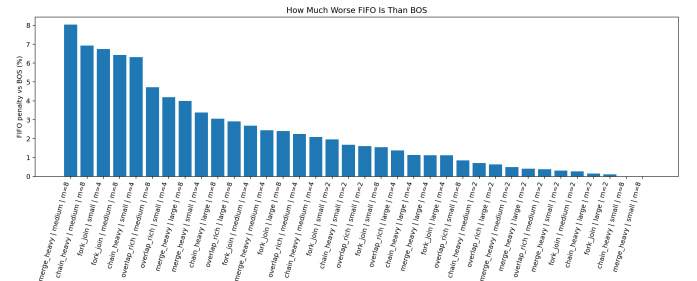


Fig. 2. Relative makespan penalty of FIFO compared with BOS. Larger positive values indicate that FIFO produces longer schedules. The results show that naive ready-order scheduling is consistently weaker than structure-aware methods.

B. Performance Relative to the Lower Bound

To evaluate schedule quality beyond raw makespan, we compare each algorithm against the theoretical lower bound

$$LB = \max\left(\frac{W}{m}, L_\delta\right).$$

Since this bound combines both workload-based and overlap-adjusted critical-chain constraints, it provides a theory-aligned reference point for empirical evaluation.

Figure 3 shows the difference between the lower-bound gap of HEFT and that of BOS. Positive values indicate cases where BOS is closer to the lower bound than HEFT, while negative values indicate the opposite. The figure shows that the gap difference is usually small, but BOS often remains at least as close to the lower bound as HEFT, and in several groups it is slightly tighter. This observation is important because it connects the empirical results with the approximation-oriented analysis developed earlier. Even when BOS does not substantially reduce raw makespan relative to HEFT, it still behaves competitively from the perspective of theoretical schedule tightness.

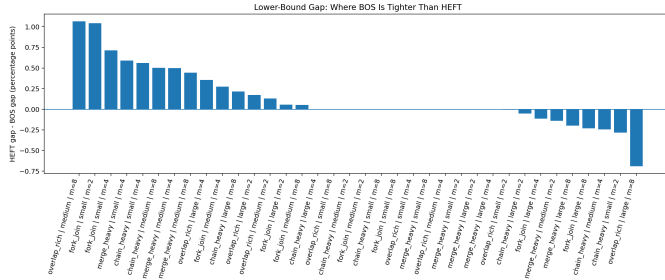


Fig. 3. Difference in lower-bound gap between HEFT and BOS. Positive values indicate that BOS is closer to the theoretical lower bound than HEFT. The small but nontrivial differences suggest that BOS remains competitive in terms of theory-aligned schedule quality.

More importantly, the small gap-to-bound values observed for both BOS and HEFT imply that the scheduling space is already tight. Therefore, the absence of a large makespan advantage for BOS should not be read as a lack of contribution. Instead, it suggests that under overlap-aware precedence constraints, multiple strong scheduling mechanisms may converge to similarly high-quality schedules.

C. Decision-Process Divergence Between BOS and HEFT

Although BOS and HEFT often produce similar makespan, their internal decision processes differ substantially. To quantify this difference, we introduced the decision disagreement rate, which measures how often two schedulers choose different tasks under the same system state.

Figure 4 shows the disagreement rate between BOS and HEFT across all instance groups. Several groups, especially in overlap-rich and large-scale settings, exhibit high disagreement rates. This means that BOS and HEFT frequently make different dispatching choices even though their final makespan remains close. Such behavior indicates that BOS is not a

trivial re-labeling of HEFT. Rather, BOS represents a distinct scheduling regime based on event-driven bottleneck updates, whereas HEFT follows a static rank-based priority order.

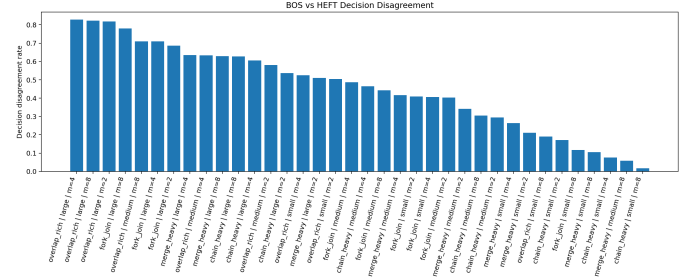


Fig. 4. Decision disagreement rate between BOS and HEFT. High values indicate that the two schedulers frequently choose different tasks under comparable states. This confirms that BOS and HEFT rely on fundamentally different dispatching mechanisms despite often producing similar makespan.

This result is one of the central findings of the study. A large decision discrepancy does not necessarily translate into a large makespan discrepancy. This suggests that under overlap-aware precedence constraints, multiple distinct scheduling strategies may converge to near-optimal solutions. From a research perspective, this strengthens the significance of BOS: its value lies not only in numerical performance, but also in providing an alternative and theoretically grounded scheduling framework.

D. Exact Validation on Small Instances

To further validate solution quality, we computed optimal schedules for small instances using the branch-and-bound solver described in Section IV. Figure 5 reports the average gap to optimality for BOS, HEFT, and FIFO on the exact benchmark families.

The results show that BOS reaches zero optimality gap on all tested small families. HEFT also performs strongly, but is not uniformly optimal in every family. FIFO, by contrast, remains consistently worse. These findings provide two important conclusions. First, BOS is not merely a heuristic that performs well on average; on small exact instances, it can recover the optimal schedule. Second, the strong behavior of both BOS and HEFT again confirms that the main empirical challenge is not beating weak baselines, but understanding how different high-quality scheduling mechanisms behave under structured overlap constraints.

E. Discussion

Taken together, the results support the following interpretation.

First, BOS is competitive with HEFT in terms of makespan, but does not consistently dominate it in neutral benchmark settings. This is a realistic and theoretically coherent outcome, rather than a limitation of the method. Since both algorithms frequently operate close to the lower bound, the room for further improvement is naturally restricted.

Second, BOS differs fundamentally from HEFT in its scheduling logic. HEFT relies on a static critical-path-inspired

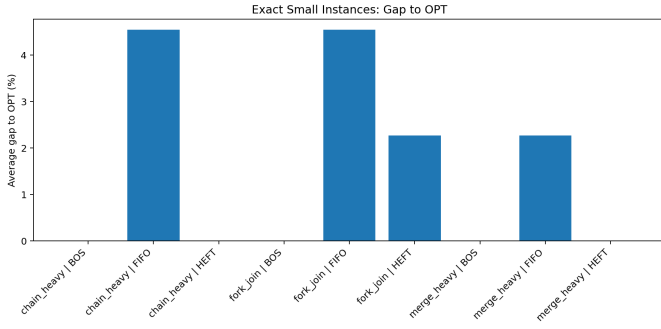


Fig. 5. Average gap to the optimal solution on small exact instances. BOS attains zero optimality gap on all tested families, while FIFO remains clearly suboptimal. This supports the validity of BOS beyond heuristic-level performance.

priority structure, whereas BOS dynamically updates bottleneck information on the residual DAG. The large disagreement rates observed in several instance groups confirm that these are genuinely different scheduling mechanisms.

Third, BOS remains clearly stronger than FIFO and reaches optimal performance on the tested exact small instances. This shows that BOS is not only theoretically motivated, but also empirically sound.

Overall, the contribution of BOS should therefore not be framed as “consistently outperforming HEFT by a large margin.” A more accurate interpretation is that BOS provides an alternative overlap-aware and event-driven bottleneck scheduling framework that remains competitive with HEFT, while being grounded in a different decision principle and supported by both lower-bound analysis and exact validation.

VI. CONCLUSION

This paper introduced BOS (Bottleneck-Oriented Scheduling), an event-driven scheduling framework designed for parallel machine systems with overlap-aware precedence constraints.

Unlike traditional list scheduling approaches such as HEFT, BOS dynamically tracks bottleneck evolution in the residual DAG and updates scheduling priorities accordingly. This enables BOS to adapt to changing system conditions rather than relying on static priority rankings.

Extensive experiments demonstrate that BOS achieves makespan performance comparable to HEFT across a wide range of benchmark instances. While BOS does not consistently outperform HEFT in absolute makespan, both methods are observed to operate close to the theoretical lower bound, indicating that the scheduling space is inherently tight.

More importantly, BOS exhibits substantial decision-process divergence from HEFT, as reflected by high disagreement rates across multiple instance groups. Despite these differences, both approaches converge to near-optimal solutions, suggesting the existence of multiple effective scheduling mechanisms under overlap-aware constraints.

In addition, BOS consistently outperforms FIFO and achieves optimal solutions in all tested small instances, further validating its effectiveness.

Overall, the contribution of this work lies not in achieving uniformly superior makespan, but in introducing a distinct and theoretically grounded scheduling paradigm that remains competitive with state-of-the-art methods while providing new insights into dynamic bottleneck-driven scheduling.

Future Work

Future work includes identifying instance classes where dynamic bottleneck tracking provides clear advantages over static priority-based methods, as well as extending BOS to heterogeneous machine environments and stochastic scheduling settings.

APPENDIX

APPENDIX A. DISCRETE-TIME FORMALIZATION OF TIME-OVERLAP ACTIVATION

This appendix provides a discrete, event-driven formulation of the time-overlap activation mechanism and its theoretical properties. No calculus or continuous-time assumptions are required; all updates occur only at discrete scheduling events.

A.1 Discrete Time Model

We consider time through a discrete event index:

$$e_1, e_2, \dots, e_K, \quad K = O(n + E),$$

where each event triggers a state update. Typical events include task completion, task activation, machine release, and material arrival. The scheduler updates its state only when such an event occurs.

At event e_k , the system state is represented by

$$\mathcal{S}(e_k) = (H(e_k), \mathcal{R}(e_k), \text{running}(e_k)),$$

where $H(e_k)$ is the priority structure, $\mathcal{R}(e_k)$ is the set of executable tasks, and $\text{running}(e_k)$ is the set of tasks currently being processed.

A.2 Discrete Time-Overlap Activation

For each dependency edge $(j, i) \in E$, let

$$\delta_{ji} \in [0, p_j]$$

denote the time overlap allowance from predecessor task j to successor task i .

At event e_k , let $x_j(e_k)$ denote the amount of processing already completed by task j by that event. Then task i becomes executable at event e_k only if

$$x_j(e_k) \geq p_j - \delta_{ji}, \quad \forall j \in \text{pred}(i),$$

and its release-time condition is also satisfied:

$$s_i \geq r_i.$$

[Discrete Time-Overlap Activation] A task i is executable at event e_k if

$$x_j(e_k) \geq p_j - \delta_{ji}, \quad \forall j \in \text{pred}(i),$$

and

$$s_i \geq r_i.$$

Equivalently, task i may start only after every predecessor has completed a sufficient absolute amount of processing under the edge-wise time-overlap rule.

When $\delta_{ji} = 0$, the above condition reduces to the classical finish-to-start AND-precedence constraint on edge (j, i) .

A.3 Effective Critical-Chain Length

Let $\mathcal{P}(i)$ be the set of all directed paths starting from task i in the precedence DAG. For a path

$$P = (i_1, i_2, \dots, i_k),$$

its overlap-adjusted effective length is defined as

$$L_\delta(P) = \sum_{t=1}^k p_{i_t} - \sum_{t=1}^{k-1} \delta_{i_t, i_{t+1}}.$$

This formulation directly captures the reduction in serial waiting caused by allowable temporal overlap along each dependency edge. When all overlap allowances are zero, the effective path length reduces to the classical path length:

$$L_\delta(P) = \sum_{t=1}^k p_{i_t}.$$

The corresponding effective critical-chain length is

$$L_\delta = \max_P L_\delta(P),$$

where the maximum is taken over all directed paths in the DAG.

A.4 Monotonicity Properties

[Monotonicity of L_δ] Let δ and δ' be two overlap settings such that

$$\delta'_{ji} \geq \delta_{ji}, \quad \forall (j, i) \in E.$$

Then

$$L_{\delta'}(P) \leq L_\delta(P), \quad \forall P,$$

and consequently,

$$L_{\delta'} \leq L_\delta.$$

For any directed path

$$P = (i_1, i_2, \dots, i_k),$$

we have

$$L_\delta(P) = \sum_{t=1}^k p_{i_t} - \sum_{t=1}^{k-1} \delta_{i_t, i_{t+1}}.$$

If $\delta'_{ji} \geq \delta_{ji}$ on every edge, then the subtracted overlap term under δ' is no smaller than that under δ . Hence

$$L_{\delta'}(P) \leq L_\delta(P)$$

for every path P . Taking the maximum over all paths yields

$$L_{\delta'} \leq L_\delta.$$

[Equivalence to Classical Precedence] If

$$\delta_{ji} = 0, \quad \forall (j, i) \in E,$$

then

$$L_\delta = L_{CP},$$

where L_{CP} is the classical critical-path length.

If all overlap allowances are zero, then no successor may start before its predecessors fully complete. Therefore,

$$L_\delta(P) = \sum_{t=1}^k p_{i_t}$$

for every directed path P , which is exactly the classical path length. Hence the maximum effective path length coincides with the classical critical-path length:

$$L_\delta = L_{CP}.$$

A.5 Interpretation

The time-overlap mechanism does not increase machine capacity and does not reduce intrinsic processing times. Its role is to reduce waiting requirements induced by strict finish-to-start precedence whenever partial-kit execution is operationally feasible.

Therefore, increasing overlap allowances can only shorten or preserve the effective downstream bottleneck length; it cannot enlarge the serial waiting requirement encoded in the precedence structure. This explains why the observed improvement is typically moderate rather than dramatic: the mechanism removes avoidable waiting gaps, but it does not alter the underlying machine environment or task durations.

A.6 Summary

This appendix formalizes time-overlap activation in a fully discrete, event-driven framework:

- no calculus or continuous-time assumptions are required;
- activation depends on edge-wise absolute processing progress, rather than predecessor completion ratios;
- effective critical-chain length L_δ is defined directly by overlap-adjusted path length;
- increasing overlap allowances monotonically decreases or preserves L_δ ;
- the classical precedence model is recovered as the special case $\delta_{ji} = 0$ on all edges.

These properties are consistent with the main formulation in the paper and confirm that the proposed time-overlap mechanism is both theoretically coherent and operationally meaningful.

REFERENCES

- [1] G. W. Plossl and J. Orlicky, *Orlicky's Material Requirements Planning*. McGraw-Hill, 1994.
- [2] J. Adams, E. Balas, and D. Zawack, "The shifting bottleneck procedure for job shop scheduling," *Management Science*, vol. 34, no. 3, pp. 391–401, 1988.
- [3] D. D. Yao, "Book review: Stochastic models of manufacturing systems," *Discrete Event Dynamic Systems*, vol. 4, no. 4, pp. 409–411, 1994.
- [4] R. W. Conway, W. L. Maxwell, and L. W. Miller, *Theory of Scheduling*. Addison-Wesley, 1967.

- [5] M. R. Garey and D. S. Johnson, *Computers and Intractability*. W. H. Freeman, 1979.
- [6] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [7] P. Brucker, A. Drexler, R. Möhring, K. Neumann, and E. Pesch, "Resource-constrained project scheduling," *European Journal of Operational Research*, vol. 112, no. 1, pp. 3–41, 1999.
- [8] R. Kolisch and S. Hartmann, "Heuristic algorithms for rcpsp," in *Project Scheduling*. Springer, 1999, pp. 147–178.
- [9] W. Herroelen and R. Leus, "Project scheduling under uncertainty," *European Journal of Operational Research*, vol. 165, no. 2, pp. 289–306, 2005.
- [10] L. P. Leach, *Critical Chain Project Management*. Artech House, 1999.
- [11] L. Lawrence, *Critical Chain Project Management*. Artech, 2014.
- [12] D. B. Shmoys, C. Stein, and J. Wein, "Improved approximation algorithms for shop scheduling," in *SODA*, 1991.
- [13] O. Svensson, "Conditional hardness of precedence constrained scheduling," 2010, pp. 745–754.
- [14] C. N. Potts and M. Y. Kovalyov, "Scheduling with batching: A review," *European Journal of Operational Research*, vol. 120, no. 2, pp. 228–249, 2000.
- [15] R. H. Möhring, M. Skutella, and F. Stork, "Scheduling with and/or precedence constraints," *SIAM Journal on Computing*, vol. 33, no. 2, pp. 393–415, 2004.
- [16] M. Gómez Sánchez, E. Lalla-Ruiz, A. Fernández Gil, C. Castro, and S. Voß, "Resource-constrained multi-project scheduling problem: A survey," *European Journal of Operational Research*, vol. 309, no. 3, pp. 958–976, 2023.
- [17] M. Aghileh, A. Tereso, F. Alvelos, and M. O. M. Lopes, "Multi-project scheduling with uncertainty and resource flexibility," *Algorithms*, vol. 18, no. 6, p. 314, 2025.
- [18] M. Urgo, G. Lanza, R. Vrabčič, and D. Gyulai, "Future-proof production scheduling and control," *CIRP Annals*, vol. 74, no. 2, pp. 971–991, 2025.
- [19] M. E. Kanakis, R. Khalili, and L. Wang, "Machine learning for computer systems and networking: A survey," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–36, 2022.
- [20] J. Orlicky, *Material Requirements Planning*. McGraw-Hill, 1975.
- [21] T. Brockhoff, P. Mavrothalassitis, E. Bakopoulos, N. Nikolakis, and D. Mourtzis, "Analyzing multi-level bom-structured event data," in *Process Mining Workshops*, 2022, pp. 47–59.
- [22] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*. Springer, 2008.
- [23] W. J. Hopp and M. L. Spearman, *Factory Physics*. Waveland Press, 2011.
- [24] C. Roser, M. Nakano, and M. Tanaka, "Shifting bottleneck detection," in *Proceedings*, 2003.
- [25] J. B. Kidd and Y. Monden, "Toyota production system," *Journal of the Operational Research Society*, vol. 46, p. 669, 1993.
- [26] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [27] L. A. Hall, "Approximability of flow shop scheduling," *Mathematical Programming*, vol. 82, no. 1, pp. 175–190, 1998.
- [28] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
- [29] C. N. Potts and L. N. Van Wassenhove, "Integrating scheduling and batching decisions for manufacturing systems," *European Journal of Operational Research*, vol. 56, no. 2, pp. 204–217, 1992.
- [30] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *HotNets*, 2016.
- [31] H. Mao, M. Schwarzkopf, S. B. Venkatakrisnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *SIGCOMM*, 2019.
- [32] K. Alexopoulos *et al.*, "Deep reinforcement learning for selection of dispatch rules," *Applied Sciences*, vol. 15, no. 1, p. 232, 2025.